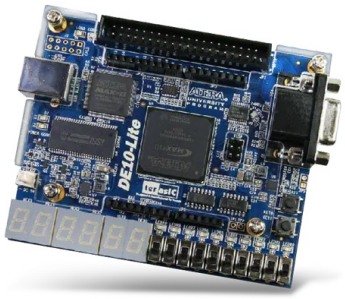
instituto superior de engenharia de lisboa

Licenciatura em Engenharia Informática, Redes e Telecomunicações



RELATÓRIO DO 1º TRABALHO

Circuitos Combinatórios

Trabalho realizado por:

55998 Diogo Câncio

55948 Tiago Loreto

Turma: LEIRT

Docente: Rui António Duarte

Lógica e Sistemas Digitais

2026 / 2027 inverno

Índice

Índice **i**

1 Objetivo **1**

2 Descrição do Circuito a Projetar **2**

3 Desenvolvimento do Projeto **3**

3.1 Expressões lógicas das funções P e Z (Módulo PnZ4) 3

3.2 Esquema lógico dos blocos PnZ4 4

3.3 Expressão lógicas da função R (Módulo PnZ4) 5

3.4 Esquema lógico do bloco Par/Ímpar 5

3.5 Circuito do PnZ10 em VHDL 6

3.6 Simulação 7

3.7 Implementação 8

3.8 Validação 8

4 Resultados e Discussão **9**

5 Conclusão **10**

6 VHDL **11**

6.1 PnZ10.vhd 11

6.2 PnZ4.vhd 13

6.3 ParImpar.vhd 14

7 Atribuição de Pinos **15**

1 Objetivo

Descrever um circuito combinatório com VHDL estrutural, simular e implementar o circuito na placa de desenvolvimento DE10-Lite da Intel.

2 Descrição do Circuito a Projetar

Pretende-se implementar um circuito que verifique se o número de entradas ativas é par ou ímpar, dependendo do sinal de entrada (PnI) e se pelo menos uma das entradas está ativa (não Zero). O diagrama do circuito (PnZ10) é apresentado na Figura 1, sendo utilizadas 10 entradas, correspondentes a 10 interruptores (switches) da placa DE10-Lite, um sinal de entrada (PnI), correspondente a um botão de pressão e 2 saídas mapeadas em dois LEDs para indicação dos sinais de Paridade (P) e não Zero (nZ).

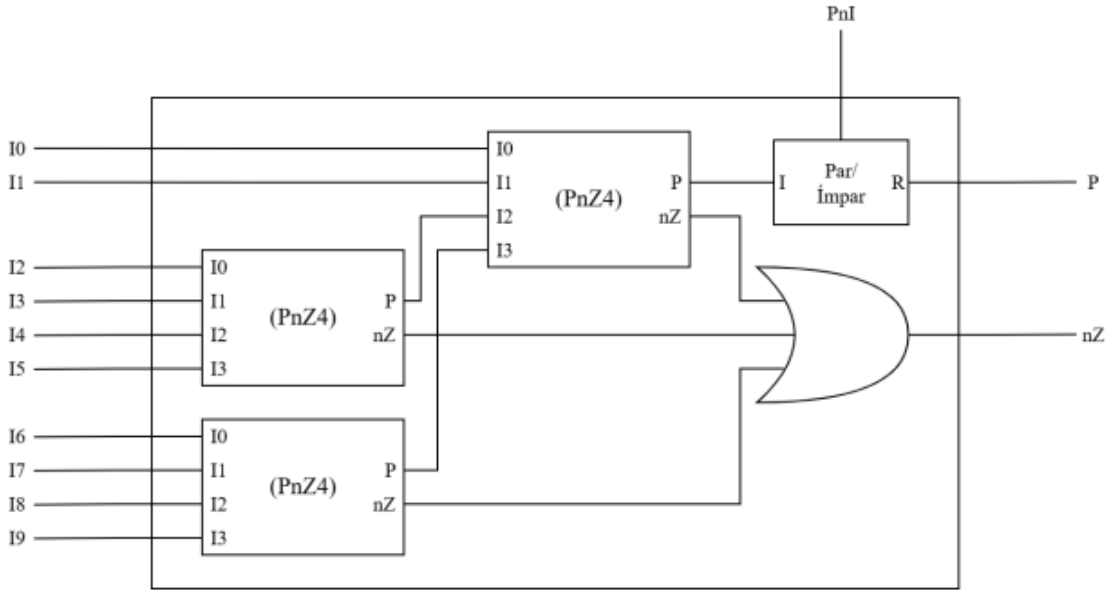


Figura 1: Diagrama do circuito PnZ10

O circuito deve verificar se o número de entradas I (I0, I1, I2, I3, I4, I5, I6, I7, I8 e I9) ativas é par ou ímpar e através do seletor de paridade par/ímpar (PnI) apresentar o resultado na saída P. Quando o sinal PnI for '0', a saída P deverá estar ativa quando o resultado é ímpar. Do mesmo modo, quando o sinal PnI assumir o valor lógico '1', a saída P deverá estar ativa quando o resultado é par. Relativamente à deteção de não zero, a saída nZ deve ficar ativa quando pelo menos uma das entradas estiver ativa e deve ficar inativa sempre que todas as entradas estejam inativas.

3 Desenvolvimento do Projeto

3.1 Expressões lógicas das funções P e Z (Módulo PnZ4)

Para este exercício, iremos usar os pins I0, I1, I2, I3, para, segundo o enunciado, determinar a função Z, que "Ativa quando pelo menos uma das entradas estiver vazia", e para a função P, que "Quando o sinal for '0', a saída P deverá estar ativa quando o resultado for ímpar", da mesma forma que quando o resultado for '1', a saída estar ativa quando o resultado for par.

Tabela 1: Quadro de sinais do módulo PnZ4

I_0	I_1	I_2	I_3	P	Z
0	0	0	0	0	0
0	0	0	1	1	1
0	0	1	0	1	1
0	0	1	1	0	1
0	1	0	0	1	1
0	1	0	1	0	1
0	1	1	0	0	1
0	1	1	1	1	1
1	0	0	0	1	1
1	0	0	1	0	1
1	0	1	0	0	1
1	0	1	1	1	1
1	1	0	0	0	1
1	1	0	1	1	1
1	1	1	0	1	1
1	1	1	1	0	1

$$Z = I_0 + I_1 + I_2 + I_3$$

$$P = ((I_0 \oplus I_1) \oplus I_2) \oplus I_3 \ = I_0 \oplus I_1 \oplus I_2 \oplus I_3$$

3.2 Esquema lógico dos blocos PnZ4

Para desenhar, utilizei o meu tablet como ferramenta, que pela demonstração ilustro exatamente como seria dentro do sistema lógico dos blocos PnZ4.

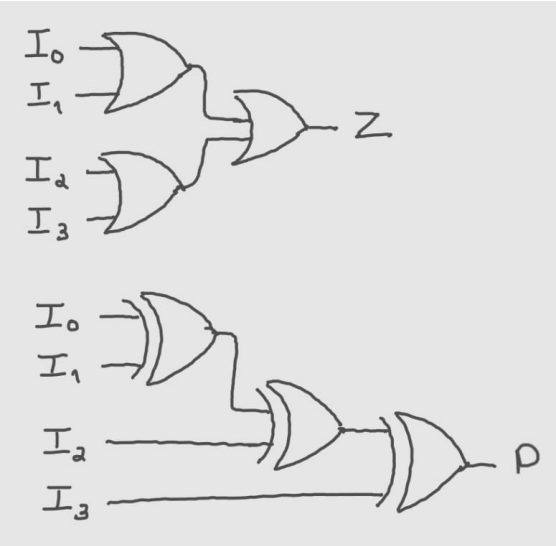


Figura 2: Sistema lógico dos blocos PnZ4

3.3 Expressão lógicas da função R (Módulo PnZ4)

Para este exercício, iremos utilizar os sinais I e PnI para determinar a função R, em que sempre que o número total de bits ativos for ímpar, a saída deverá assumir '1', caso contrário, quando for par, a saída deverá assumir o valor lógico '0'.

Tabela 2: Quadro de sinais do módulo PnZ4

I	PnI	R
0	0	0
0	1	1
1	0	1
1	1	0

$$R = I \oplus PnI$$

3.4 Esquema lógico do bloco Par/Ímpar

Novamente, para desenhar utilizei o meu tablet e ilustração apresento como seria dentro do sistema lógico dos blocos Par/Ímpar.

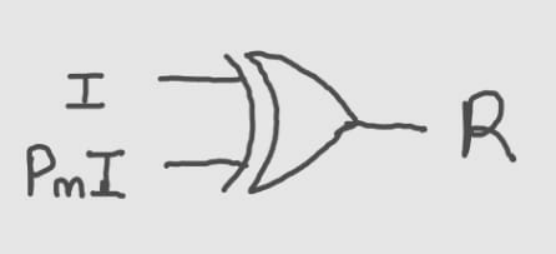


Figura 3: Sistema lógico do bloco Par/Ímpar

3.5 Circuito do PnZ10 em VHDL

O circuito que iremos descrever segue a Figura 1 com três entidades distintas a qual apresento por esta imagem retirada pelo RTL Viewer do Quartus:

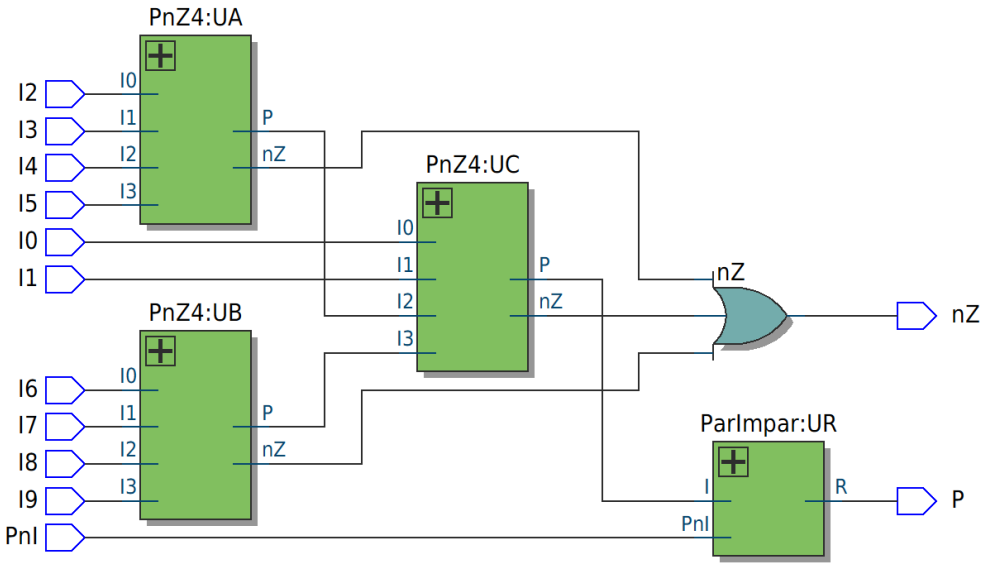


Figura 4: RTL Viewer do circuito PnZ10

- **PnZ4**: Que faz o trabalho de um elemento XOR entre as 4 entradas;
  - Foram usados 3 PnZ4 em que:
    - UA (entradas I2 a I5)
    - UB (I6 a I9)
    - UC (I0, I1 e as paridades de UA e UB).
  - A paridade de UC é a das 10 entradas e vai para o ParImpar, que dá a saída P.
  - A saída nZ junta os nZ dos três PnZ4 com um OR.
  - Só o do UC não chegava: se I2 e I3 estiverem ativas, a paridade do UA fica a 0.
- **ParImpar**: Usa o PnI para definir se é par ou ímpar;
- **PnZ10**: O topo, que acaba por ligar tudo.

3.6 Simulação

Usei o ModelSim através do Quartus. Escrevi-o com que o PnI a 0 e a 1 verificam as saídas sozinhos.

Tabela 3: Quadro de resultados de saídas

Entradas ativas	P com PnI=0	P com PnI=1	nZ
0	0	1	0
1	1	0	1
2	0	1	1
3	1	0	1
10	0	1	1

3.7 Implementação

O projeto foi criado no Quartus DE-10 lite, com a criação do ficheiro PnZ10 no topo da hiearquia.

Ligamos as entradas aos interruptores, o PnI ao botão KEY0 (Quando primido, atribui 0, caso contrário 1), e o P e nZ aos dois LEDs.

3.8 Validação

Compilamos e não programamos a placa, pois de momento de entrega deste trabalho, foi-nos informado que fariamos após a sua entrega. Em contra-partida, na imagem seguinte, explicamos a atribuição de pins que fizemos de uma forma física.

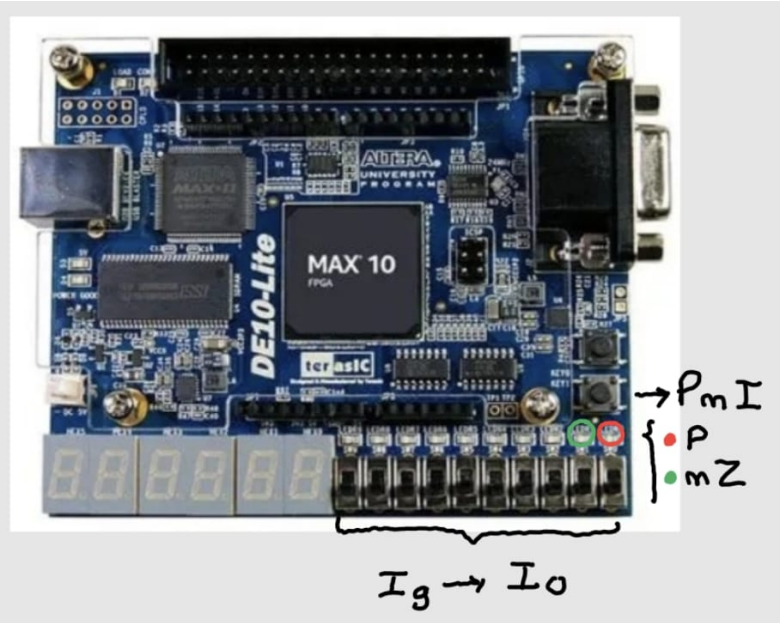


Figura 5: Representação da atribuição física

4 Resultados e Discussão

A simulação (2048 casos) funcionou após algumas alterações.

O nZ só fica desligado com todas as entradas a 0, e o P inverte quando se muda o PnI.

Houve dificuldades quanto á compreenção dos objetivos do trabalho, tal como no seu armazenamento correto e preparação para envio, o que nos levou a ter de quase refazer o trabalho do zero duas vezes.

Foi também necessário recorrer a fontes de apoio adicionais para esclarecer alguns conceitos e garantir a correta implementação do circuito.

5 Conclusão

Este trabalho consistiu na realização do circuito combinatório PnZ10, e, para isso, utlizamos três módulos PnZ4 e de um módulo Par/Ímpar, em que a sua construção consistiu num projeto em VHDL estrutural,

A validação do funcionamento foi efetuada através de simulação com recurso a um testbench e, posteriormente, através da implementação na placa DE10-Lite.

Este trabalho permitiu o reforço de conhecimentos não só na construção de uma estrutura VHDL, como também o desbloqueio para a compreensão da matéria inicial da cadeira LSD (Lógica e Sistemas Digitais).

6 VHDL

6.1 PnZ10.vhd

```
library IEEE;
use IEEE.std_logic_1164.all;

entity PnZ10 is
        port (
                I0, I1, I2, I3, I4, I5, I6, I7, I8, I9 : in
                ↪  std_logic;
                PnI : in  std_logic;
                P   : out std_logic;
                nZ  : out std_logic);
end PnZ10;

architecture structural of PnZ10 is

component PnZ4 is
port (
        I0, I1, I2, I3 : in  std_logic;
        P, nZ: out std_logic);
end component;

component ParImpar is
        port (
                I, PnI : in  std_logic;
                R: out std_logic);
end component;

signal P_a, P_b, P_c   : std_logic;
signal nZ_a, nZ_b, nZ_c : std_logic;

begin
```

```
        UA : PnZ4 port map (
                I0 => I2, I1 => I3,
                I2 => I4, I3 => I5,
                P => P_a, nZ => nZ_a);

        UB : PnZ4 port map (
                I0 => I6, I1 => I7,
                I2 => I8, I3 => I9,
                P => P_b, nZ => nZ_b);

        UC : PnZ4 port map (
                I0 => I0,
                I1 => I1,
                I2 => P_a, I3 => P_b,
                P => P_c, nZ => nZ_c);

        UR : ParImpar port map (
                I => P_c,
                PnI => PnI,
                R => P);

        nZ <= nZ_a or nZ_b or nZ_c;

end structural;
```

6.2 PnZ4.vhd

```
library IEEE;
use IEEE.std_logic_1164.all;

entity PnZ4 is
        port (
                I0, I1, I2, I3 : in  std_logic;
                P  : out std_logic;    -- '1' if odd number of
                ↪  active inputs
                nZ : out std_logic);  -- '1' if at least one input
                ↪  is active
end PnZ4;

architecture logic of PnZ4 is
begin
        P  <= I0 xor I1 xor I2 xor I3;
        nZ <= I0 or I1 or I2 or I3;
end logic;
```

6.3 ParImpar.vhd

```
library IEEE;
use IEEE.std_logic_1164.all;

entity ParImpar is port (
  I: in  std_logic;           -- parity (1 = odd) of the inputs
  PnI : in  std_logic;  -- 0: P active if odd | 1: P active if even
  R   : out std_logic);
end ParImpar;

architecture logic of ParImpar is
begin
        R <= I xor PnI;
end logic;
```

7 Atribuição de Pinos

Entradas	**pino**
I0	C10
I1	C11
I2	D12
I3	C12
I4	A12
I5	B12
I6	A13
I7	A14
I8	B14
I9	F15
Saídas	**LED's**
P	A8
nZ	A9
Manipulações	**Botões**
PnI	KEY0

Tabela 4: Atribuição de pinos